CS683 Programming Assignment II

Stride pattern prefetching, instruction-conscious L2 replacement, and their combination

Atharva Bhutada	24B0996
Divyaansh Narkhede	24B0981
Pranav Patil	24B1074
Sufiyan	24B0935

Autumn 2026

Contents

1 Experimental setup **2**

2 Task 1A: the IP-stride prefetcher **2**

3 Task 1B: Stride Pattern Prefetcher (SPP) **3**
- 3.1 Design . . . 3
- 3.2 Results . . . 4
- 3.3 Storage overhead . . . 5
- 3.4 Choice of the throttling thresholds . . . 6
- 3.5 Choice of the degrees . . . 7

4 Task 2A: Instruction-conscious L2 replacement (I-RRIP) **7**
- 4.1 Passing the instruction bit to the policy . . . 7
- 4.2 Design . . . 8
- 4.3 Results . . . 8
- 4.4 Why this combination: design ablation . . . 9
- 4.5 Hardware cost . . . 10

5 Task 2B: Dynamic I-RRIP **10**
- 5.1 Design (threshold version, `dyn_irrip.l2c_repl`) . . . 10
- 5.2 Results (threshold 0.25) . . . 11
- 5.3 Sensitivity to the threshold . . . 12
- 5.4 A second design: true set dueling (`dyn_duel.l2c_repl`) . . . 13

6 Task 3: SPP with prefetch-aware dynamic I-RRIP **15**
- 6.1 Setup . . . 15
- 6.2 How the policy handles prefetch packets . . . 15
- 6.3 Results . . . 16
- 6.4 Sensitivity to the threshold . . . 18

1 Experimental setup

We used the CS683 PA2 version of ChampSim with one core and the hashed perceptron branch predictor. The caches that matter here are:

- L1D: 48 KB, 12 ways, 64 sets, 16 MSHRs, 16-entry prefetch queue.
- L2C: 512 KB, 8 ways, 1024 sets.
- LLC: 2 MB, 16 ways.

A cache line is 64 B and a page is 4 KB. All caches except the L2C use LRU. Only the L1D can have a prefetcher. Each run uses 25M warmup instructions and then 25M simulated instructions. All numbers in this report come from the simulated part. We built every binary with `g++-7`.

ChampSim picks its random seed from the name of the folder that holds the trace. This seed decides how virtual pages map to physical pages. We always read the traces as `../traces/TraceN.gz`, in all three tasks, so every run gets the same page mapping. Because of this, the Task 1 baseline and the Task 2 LRU run are the same simulation and give the same numbers.

Metrics.

- Speedup is $\text{IPC}/\text{IPC}_{\text{baseline}}$ on the same trace. For Tasks 1 and 3 the baseline has no prefetcher and LRU at the L2C. For Task 2 the baseline is LRU at the L2C. This is the same run.
- MPKI is misses per 1000 simulated instructions, as printed by ChampSim. L2 instruction MPKI counts L2C load misses from instruction fetches. L2 data MPKI counts the rest.
- Prefetch accuracy is the L1D accuracy that ChampSim prints: useful prefetches divided by prefetches sent to the next level.
- Coverage uses the formula from the assignment:

$$\text{Coverage} = \frac{\text{L1D load misses}_{\text{baseline}} - \text{L1D load misses}_{\text{prefetcher}}}{\text{L1D load misses}_{\text{baseline}}}.$$

- When we give one number for all three traces, it is the geometric mean of the speedups.

Reproducing the results. Each task has one script (Table 1). The script builds the binaries, runs all the simulations, and makes the tables and plots used in this report. The tables are saved as `.md` and `.csv` files next to the ChampSim output files.

Task	Code	Script	Output
1A, 1B	`prefetcher/ip_stride.l1d_pref`, `prefetcher/spp.l1d_pref`	`scripts/task1_all.sh`	`results/`
2A	`replacement/irrip.l2c_repl` (+ ablation variants)	`scripts/task2_all.sh`	`results_task2/`
2B	`replacement/dyn_irrip.l2c_repl`, `replacement/dyn_duel.l2c_repl`	`scripts/task2_all.sh`	`results_task2/`
3	`replacement/dyn_irrip.l2c_repl` with `IRRIP_PF_AWARE=1`	`scripts/task3_all.sh`	`results_task3/`

Table 1: Where each task lives in the repository.

2 Task 1A: the IP-stride prefetcher

The given prefetcher is in `prefetcher/ip_stride.l1d_pref`. It has a table of 24 entries, and any IP can go in any entry. Only the low 8 bits of the IP are kept (`MASK_IP = 0xFF`), so two loads whose IPs have the same low 8 bits share one entry. Each entry stores the last cache line this IP touched, the last stride, and an LRU position.

On every L1D load, the prefetcher looks for the IP in the table.

- If the IP is not found, it replaces the LRU entry. It saves the current line with stride 0 and sends no prefetch.

- If the IP is found, it computes the stride: the number of cache lines between the saved line and the current line. A stride of 0 is ignored. If the new stride is the same as the saved stride, the pattern has repeated once. The prefetcher then sends lines $A + s, A + 2s, A + 3s$ (degree 3).
- If a prefetch address falls in a different 4 KB page, the loop stops. The prefetcher does not know which physical page comes next.
- If the L1D MSHR is less than half full, the line is brought into the L1D (`FILL_L1`). If not, it is only brought into the L2 (`FILL_L2`). This stops a busy L1D from getting even more misses.
- At the end, the entry saves the new line and stride and becomes the most recently used entry.

This design is cheap and works well when the stride is constant. It has three problems, and Task 1B fixes them. First, it trusts a stride after seeing it only twice, so an IP with random strides causes useless prefetches. Second, it cannot learn a sequence of strides. A pattern like $(+1, +2, +2, +2, +3)^n$ rarely gives two equal strides in a row, so it gets almost no prefetches. Third, the degree is always 3, even when most prefetches are wrong.

3 Task 1B: Stride Pattern Prefetcher (SPP)

3.1 Design

Our SPP is in `prefetcher/spp.l1d_pref`. It has three tables and a small unit that controls the degree (the throttle).

IP table (64 entries, direct mapped). The index is `(ip ^ (ip >> 6)) % 64` and the tag is 8 bits of `ip >> 6`. Each entry stores the last line address, the last stride, a 2-bit confidence counter and an 8-bit signature. If the tag does not match, the entry is replaced and no prefetch is sent. A stride of more than 63 lines (up or down) cannot stay inside one page, so it clears the entry's history.

Uniform strides. The confidence counter goes up by one when the new stride is the same as the last one. Otherwise it goes down by one. When it reaches 3 (three repeats in a row), we treat the IP as having a constant stride. The prefetcher then sends $A + s, A + 2s, \ldots, A + d\,s$, where d is the current degree. This is the same idea as IP-stride, but it needs more repeats before it trusts a stride.

Non-uniform strides. Every IP keeps a signature of its recent strides:

$$\text{sig} \leftarrow (\text{sig} \ll 2) \oplus (\text{stride} \,\&\, \text{0x7F}) \pmod{256}.$$

The newest stride goes into the low 7 bits. Older strides move left by two bits each time, so the signature holds roughly the last three or four strides. The signature is the index into a 256-entry pattern table. All IPs share this table. Each entry stores the stride that came after this signature last time, and a 2-bit confidence.

On every access we first update the entry of the old signature with the stride we just saw. If it is the same stride, confidence goes up. If it is different, confidence goes down. If confidence is 0, the entry is replaced. Then we add the new stride to the signature.

If the IP is not in uniform mode, we predict with the pattern table. We look up the current signature. If its confidence is at least 2, we prefetch $A +$ stride, add that stride to a copy of the signature, and look up again. We do this up to d times, and stop at the first entry with low confidence.

Take the example from the assignment, $(+1, +2, +2, +2, +3)^n$. The points "after $+1, +2$", "after $+1, +2, +2$" and "after $+2, +2, +2$" all have different signatures. So the table learns that the first two are followed by $+2$ and the third by $+3$. For this IP the confidence counter never reaches 3, because no stride repeats three times in a row. So the IP stays in pattern mode.

Priority and fallback. We check for a uniform stride first, as the assignment asks. When a constant stride breaks, one wrong stride drops the confidence from 3 to 2. On its next access the same IP switches to the pattern table. The pattern table keeps learning while an IP is in uniform mode, so it is already trained when the switch happens.

Page boundary. In both modes, every prefetch address is checked against the 4 KB page of the access that caused it. We stop at the first address outside that page. The L1D prefetcher works with virtual addresses, so this is a check on the virtual page. ChampSim translates each prefetch to a physical address later.

Prefetch filter and accuracy checker. A 256-entry direct-mapped filter (a valid bit and an 8-bit tag) remembers lines that were prefetched into the L1D but not used yet.

- When a prefetched line arrives in the L1D, we set its filter entry and add one to the *issued* counter.
- When a demand load uses a line that is in the filter, we add one to the *useful* counter and clear the entry. This way each line is counted only once.
- When a line is evicted from the L1D, we clear its entry.
- Before sending a prefetch, we skip it if the line is already in the filter. This stops us from sending the same prefetch twice.

The `operate` function gets virtual addresses. So the filter must also use virtual addresses when lines arrive or leave. ChampSim gives these to `cache_fill` as `v_addr` and `v_evicted_addr`. If we used the physical addresses there instead, the lookups would never match the entries.

Throttling. After every 256 prefetches that arrive in the L1D (one epoch), we compute accuracy = useful/issued. We set the degree from Table 2 and reset both counters. The degree starts at 4.

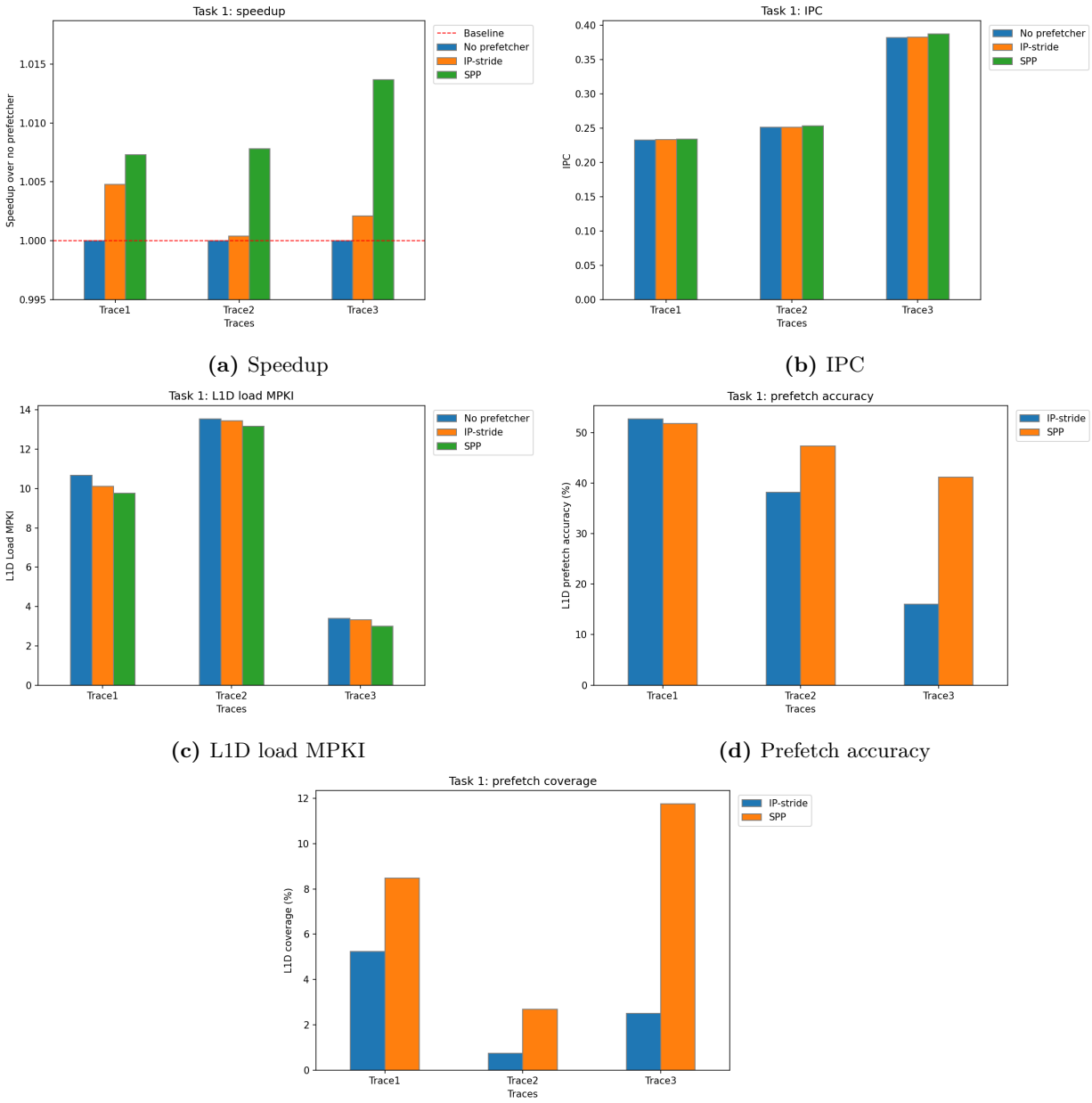
Epoch accuracy	Degree
$\geq 75\%$	6
50% to 75%	4
25% to 50%	2
$< 25\%$	1

Table 2: Accuracy ranges and the prefetch degree used in each.

3.2 Results

Trace	Prefetcher	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage (%)
Trace1	none	0.2323	1.0000	10.678	n/a	n/a
	IP-stride	0.2335	1.0048	10.119	52.71	5.24
	SPP	**0.2340**	**1.0073**	**9.771**	51.79	**8.49**
Trace2	none	0.2512	1.0000	13.538	n/a	n/a
	IP-stride	0.2513	1.0004	13.438	38.18	0.74
	SPP	**0.2531**	**1.0078**	**13.173**	**47.32**	**2.69**
Trace3	none	0.3821	1.0000	3.403	n/a	n/a
	IP-stride	0.3829	1.0021	3.318	15.99	2.51
	SPP	**0.3873**	**1.0137**	**3.003**	**41.14**	**11.76**

Table 3: Task 1 results. LRU at the L2C in all runs. Geometric-mean speedup: IP-stride 1.0024, SPP 1.0096.



(a) Speedup

(b) IPC

(c) L1D load MPKI

(d) Prefetch accuracy

(e) Coverage

Figure 1: Task 1 plots.

SPP is faster than IP-stride on every trace, and it covers more misses on every trace. The biggest gap is on Trace3: a 1.37% speedup against 0.21%. There SPP removes 11.8% of the L1D load misses. On Trace2, IP-stride does almost nothing (0.74% coverage). SPP still covers 2.7% of the misses, with higher accuracy. This suggests that Trace2 has access patterns that repeat but do not have a constant stride. On Trace1 both prefetchers have about the same accuracy, but SPP sends more prefetches and covers 8.5% of misses against 5.2%.

3.3 Storage overhead

Table 4 lists every field with the number of bits that hardware would need. In a few places the C code uses a bigger type, for example `uint8_t` for a 2-bit counter. We count the bits needed for the values a field can actually hold.

Structure	Field	Why this width	Bits
IP table entry	valid		1
	tag	8 bits of `ip >> 6`	8
	last line	low 16 bits of the line address; enough to compute strides of up to ±63	16
	last stride	signed, −63 to +63	7
	confidence	0 to 3	2
	signature		8
	entry total		*42*
	64 entries		*2688*
Pattern table entry	next stride	signed, −63 to +63	7
	confidence	0 to 3	2
	256 entries × 9 bits		*2304*
Prefetch filter entry	valid		1
	tag	line address / 256, 8 bits	8
	256 entries × 9 bits		*2304*
Throttle	issued counter	counts to the 256-prefetch epoch	9
	useful counter	9 bits, saturating; any value above the issued count already means degree 6	9
	degree	values 1, 2, 4, 6	3
	total		*21*
Total			**7317 bits ≈ 915 bytes**

Table 4: SPP storage, field by field. The running totals `totalIssued` and `totalUseful` only feed the end-of-run printout and are not part of the hardware.

The whole prefetcher needs less than 1 KB, about 1.9% of the 48 KB L1D. It is small because all IPs share one pattern table. If each IP had its own history, we would need one pattern table per IP.

3.4 Choice of the throttling thresholds

We measure accuracy over epochs of 256 prefetches. This is short enough to follow changes in the program (Trace1 has about 435 epochs in one run). It is also long enough that a short burst of bad prefetches does not change the degree by itself. We split accuracy into four equal ranges, at 25%, 50% and 75%. We did not try other thresholds. We picked these ranges by what each one means, as explained below. Table 5 shows that all four ranges are used.

- Below 25%, three out of four prefetches are wasted. Each one takes an L1D way, an MSHR and memory bandwidth. So we drop to degree 1. We do not turn the prefetcher off: one prefetch at a time keeps putting new lines in the filter, so the next epoch can tell if the pattern has come back.
- From 25% to 50%, the pattern is weak but real. Degree 2 still runs a little ahead without adding too many bad lines.
- From 50% to 75%, most prefetches are used. We use degree 4, the same as the starting value.
- At 75% or more, the pattern is reliable. The only risk left is that prefetches arrive late, so we run as far ahead as we allow.

To see what the throttle does, we ran a copy of the prefetcher with one extra counter: how many epochs it spent at each degree. Nothing else was changed, and the IPC of that run is the same as the committed run.

Trace	degree 1	degree 2	degree 4	degree 6
Trace1	6	125	185	119
Trace2	30	36	71	90
Trace3	38	53	85	93

Table 5: Number of 256-prefetch epochs spent at each degree.

Every trace uses all four degrees, so the throttle follows the program instead of staying at one value. Over the whole run, the throttle measures 62% accuracy on Trace1, 64% on Trace2 and 59% on Trace3. These are different from ChampSim's L1D accuracy in Table 3. The throttle counts the whole run, including warmup, and only counts prefetches that arrived in the L1D. ChampSim counts only the simulated part, and divides by the prefetches it sent to the L2.

3.5 Choice of the degrees

The degree decides how far ahead the prefetcher runs. A higher degree makes prefetches more likely to arrive on time, but also brings in more lines that may not be used.

- The maximum is 6. A 4 KB page has 64 lines. With a stride of a few lines, a degree much higher than 6 would often reach the end of the page and be cut short anyway. The L1D also has only 16 MSHRs and a 16-entry prefetch queue. Six prefetches is less than half the queue, so there is still room for prefetches from other IPs. Above 6 we would mostly fill the queue faster than the L2 can send lines back.
- Degree 4 is used for the middle range and is also the starting value. Between 50% and 75% accuracy, more than half of the lines are used, so running ahead pays off. But a quarter or more are still wasted, so we stay below 6. Starting at 4 means the first epoch measures accuracy at a middle setting, not at one of the extremes.
- Degrees 2 and 1 are for low accuracy. When accuracy is low, each extra line is more likely to be wasted than used. Halving the degree at each step keeps the number of wasted lines in line with the measured accuracy. We never go down to 0, so the prefetcher can recover by itself when accuracy goes back up.
- With 1, 2 and 4, each step down halves the degree. At the top we use 6 instead of 8 because of the page size and the queue size, as explained above.

4 Task 2A: Instruction-conscious L2 replacement (I-RRIP)

4.1 Passing the instruction bit to the policy

We added a ninth parameter, `uint8_t isInstruction`, to `update_replacement_state`. To do this we changed:

- the function-pointer type and the nine declarations in `inc/cache.h`;
- the function in every `replacement/*` file;
- the eight calls in `src/cache.cc` (fills, writeback hits and fills, read hits, prefetch hits, and the DTLB prefetch buffer);
- the three BTB calls in `src/ooo_cpu.cc`.

Every call in `cache.cc` passes the packet's `instruction` field. The BTB calls pass 0.

In the policy, a request is an instruction line when the packet is a `LOAD` or `PREFETCH` and has `instruction` set. `fill_cache()` uses the same rule to set the `instruction` bit of each L2 block. So the bit that the victim search reads always matches what the update function sees. Page-walk packets from the ITLB also have `instruction = 1`, but ChampSim stores them as translation blocks, so we treat them as data.

4.2 Design

Like SRRIP, the policy keeps a 2-bit RRPV for each line. RRPV 0 means the line should be used again soon. RRPV 3 means it will not be used for a long time, so it can be evicted.

Part	SRRIP	I-RRIP	Modified?
Insertion	every line at RRPV 2	instruction lines at 0, data lines at 2	yes
Promotion	every hit to RRPV 0	instruction hit to 0, data hit to 1	yes
Eviction	first line found at RRPV 3	a data line at RRPV 3 first; an instruction line only if no data line is at 3	yes

Table 6: I-RRIP compared with SRRIP.

Why instructions get priority. When an instruction fetch misses in the L2, the front end stops. No new instructions enter the ROB until the line comes back, so the core has nothing else to do while it waits. A data miss is different. The core can keep running other independent instructions and loads, which hides part of the wait. So one instruction miss costs more cycles than one data miss. On Trace1 and Trace2, more than half of all L2 load misses under LRU are instruction misses (10.3 of 17.7 MPKI and 18.9 of 27.4 MPKI). There is a lot to gain.

Insertion. A new instruction line gets RRPV 0. It must age three times before it can be evicted. A new data line gets RRPV 2, as in SRRIP, and can be evicted after aging once. In server code, the same functions run again and again, but the gap between uses is too big for the L1I. Starting at 0 keeps these lines in the L2 long enough to be used again.

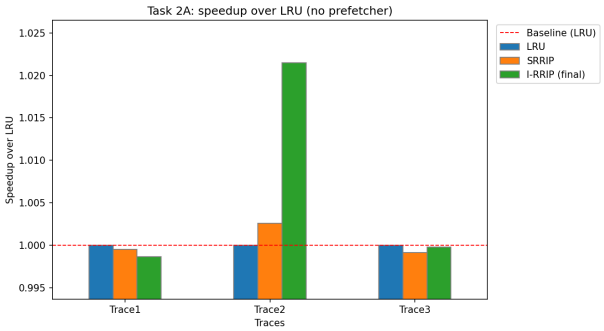
Promotion. In SRRIP, one hit moves a data line to RRPV 0, the same level as the most protected instruction lines. But the L1D already catches most short-term data reuse. So one L2 hit on a data line tells us less about future use than a hit on code. We move data hits only to RRPV 1. A reused data line then survives two aging steps instead of one, so reuse is still rewarded. But it never ranks above an instruction line of the same age.

Eviction. Aging adds one to every line in the set at the same time, so several lines often reach RRPV 3 together. SRRIP then evicts the one in the lowest way, which is just chance. In that case we evict a data line. If every line at RRPV 3 is an instruction line, we evict one of them. This way a set full of code still works and no line stays forever. Empty ways are always used first.

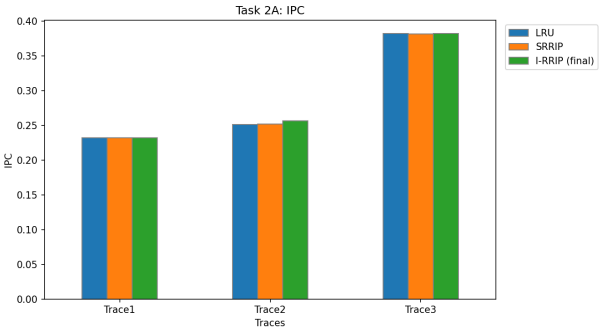
4.3 Results

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace1	LRU	0.2323	1.0000	17.696	10.338	7.358
	SRRIP	0.2322	0.9995	17.639	10.562	7.077
	I-RRIP	0.2320	0.9987	17.757	**9.837**	7.920
Trace2	LRU	0.2512	1.0000	27.357	18.853	8.504
	SRRIP	0.2518	1.0026	25.746	17.781	7.965
	I-RRIP	**0.2566**	**1.0215**	**22.414**	**13.094**	9.320
Trace3	LRU	0.3821	1.0000	3.954	1.124	2.830
	SRRIP	0.3817	0.9991	4.039	1.197	2.842
	I-RRIP	0.3820	0.9998	**3.887**	**0.998**	2.889

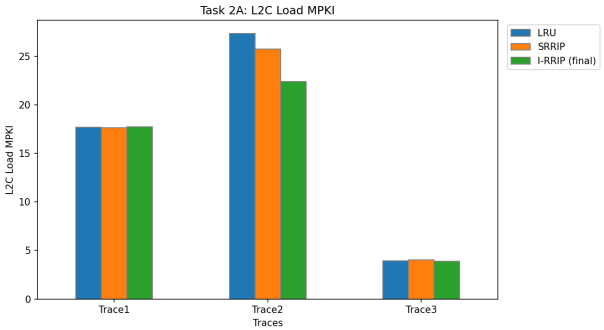
Table 7: Task 2A results (no prefetcher). Geometric-mean speedup over LRU: SRRIP 1.0004, I-RRIP 1.0066.



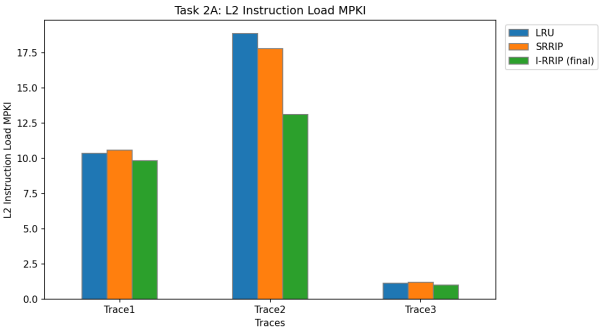
(a) Speedup



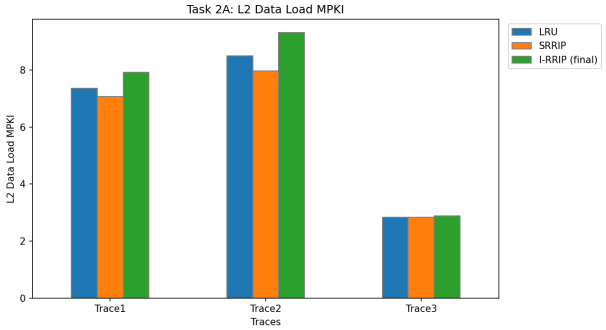
(b) IPC



(c) L2C load MPKI



(d) L2 instruction load MPKI



(e) L2 data load MPKI

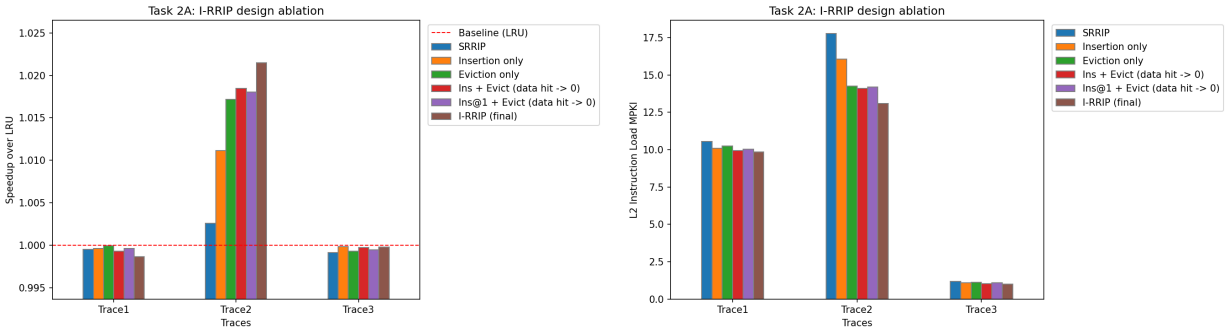
Figure 2: Task 2A plots.

4.4 Why this combination: design ablation

We built each change on its own and ran every version on all three traces with the same setup.

Variant	Insert (I/D)	Data hit	Tie-break	T1	T2	T3	Geomean	Instr.	Data
LRU	n/a	n/a	n/a	1.0000	1.0000	1.0000	1.0000	10.11	6.23
SRRIP	2 / 2	0	no	0.9995	1.0026	0.9991	1.0004	9.85	5.96
Insertion only	0 / 2	0	no	0.9996	1.0111	**0.9999**	1.0035	9.08	6.38
Eviction only	2 / 2	0	yes	**1.0000**	1.0172	0.9993	1.0055	8.54	6.20
Insertion + eviction	0 / 2	0	yes	0.9993	1.0185	0.9997	1.0058	8.37	6.59
Insert at 1 + eviction	1 / 2	0	yes	0.9996	1.0180	0.9995	1.0057	8.44	6.42
I-RRIP (final)	0 / 2	**1**	yes	0.9987	**1.0215**	0.9998	**1.0066**	**7.98**	6.71

Table 8: Ablation. T1 to T3 are speedups over LRU; the last two columns are the average L2 instruction and data load MPKI over the three traces.



(a) Speedup

(b) L2 instruction load MPKI

Figure 3: Task 2A ablation plots.

We learned three things. First, the eviction rule does most of the work. On its own it gives 1.7% on Trace2, while the insertion change alone gives 1.1%. This is because aging moves the whole set at once, and the eviction rule decides which line leaves in exactly those cases. Second, inserting instructions at 1 instead of 0 makes almost no difference once the eviction rule is there (1.0057 against 1.0058). Third, moving data hits only to RRPV 1 gives the lowest instruction MPKI of all versions (7.98) and the best geometric mean, with a 2.1% gain on Trace2.

The final design is the best version we tried, both in geometric-mean speedup and in instruction MPKI. The cost is in the data column, which is the highest of all versions. On Trace1 this loses about 0.1% of IPC compared with versions that protect data more. We accept this. Trace2, where the L2 matters most, gains the most, and on Trace1 no version beats LRU (the best one only ties it).

4.5 Hardware cost

I-RRIP uses 2 RRPV bits per line, like SRRIP: $8192 \times 2 = 16{,}384$ bits (2 KB). It also needs one bit per line to mark instruction lines, used when choosing a victim: 8192 bits (1 KB). So the extra cost over SRRIP is 1 KB, about 0.2% of the 512 KB cache. The only extra logic is one more pass in the victim search that looks for a data line at RRPV 3.

5 Task 2B: Dynamic I-RRIP

5.1 Design (threshold version, `dyn_irrip.l2c_repl`)

Leader sets. 32 of the 1024 sets always run plain SRRIP. These are the leader sets. The other 992 sets (followers) run SRRIP or I-RRIP, based on what the leaders measure. To pick the leaders, we split the sets into 32 groups of 32 sets in a row. In group g we take the set at position g, that is `set % 32 == (set / 32) % 32`. This gives sets 0, 33, 66, ..., 1023. If we took every 32nd set instead (0, 32, 64, ...), all leaders would have the same five low index bits. Then a strided access pattern could hit the leaders much more, or much less, than the rest of the cache.

Why 32 leaders. 32 is the usual number in set-dueling designs for caches of this size (DIP, DRRIP). Our leaders are 3.1% of the sets and get about that share of the L2 traffic. This gives 25 to 73 decisions per run on these traces. With fewer leaders, each epoch would take longer or the ratio would be noisier. With more leaders, a bigger part of the cache would always run SRRIP, even when I-RRIP is better. We can already see this cost in the results below.

What the leaders count. Every hit and every fill in a leader set counts as an access. Every fill also counts as a miss, because a fill is a miss whose data has come back. We keep these counts for all lines and also for instruction lines only. The counts for data lines are the difference. We do not count writebacks. The core does not wait for an L1D eviction that goes to the L2. Counting it would make the data share look bigger without telling us anything about stalls.

Decision. After every 1024 counted leader events we compute the formula from the assignment,

$$\text{ratio} = \frac{\text{instr. accesses} + \text{instr. misses}}{\text{total accesses} + \text{total misses}},$$

If the ratio is above the threshold, the followers use I-RRIP for the next epoch. If not, they use SRRIP. Then we halve all four counters. This way old behaviour fades out slowly, and one unusual epoch cannot change the mode by itself.

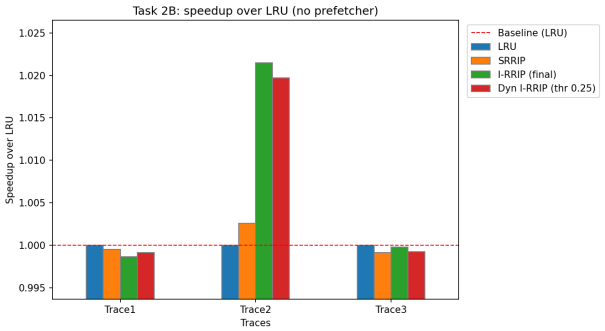
The leaders run SRRIP, so the ratio shows how much of the L2 traffic is code when code is not protected. A miss is counted twice (once as an access and once as a miss). So a program whose instruction lines keep missing gets a higher ratio than one whose instruction lines already hit. In short, the ratio is high when protecting code can help, and low when the L2 is busy with data.

Hardware cost. On top of I-RRIP, the policy needs four 12-bit counters (halving every 1024 events keeps each count below 2048), a 10-bit epoch counter, one mode bit and an 8-bit threshold register. That is 67 bits. To check if a set is a leader, we compare two 5-bit parts of the set index. This needs no storage. In hardware we would not divide. We would check $256 \cdot (\text{instr. acc.} + \text{instr. miss}) > T \cdot (\text{total acc.} + \text{total miss})$, with T in units of $1/256$.

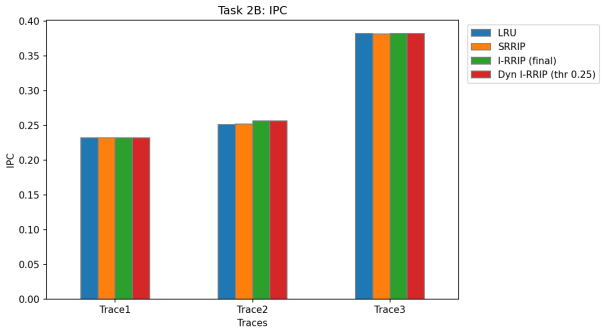
5.2 Results (threshold 0.25)

Trace	Policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace1	LRU	0.2323	1.0000	17.696	10.338	7.358
	SRRIP	0.2322	0.9995	17.639	10.562	7.077
	I-RRIP	0.2320	0.9987	17.757	9.837	7.920
	Dynamic I-RRIP	0.2321	0.9991	17.740	9.845	7.895
Trace2	LRU	0.2512	1.0000	27.357	18.853	8.504
	SRRIP	0.2518	1.0026	25.746	17.781	7.965
	I-RRIP	0.2566	1.0215	22.414	13.094	9.320
	Dynamic I-RRIP	0.2561	1.0197	22.752	13.465	9.287
Trace3	LRU	0.3821	1.0000	3.954	1.124	2.830
	SRRIP	0.3817	0.9991	4.039	1.197	2.842
	I-RRIP	0.3820	0.9998	3.887	0.998	2.889
	Dynamic I-RRIP	0.3818	0.9993	3.976	1.096	2.880

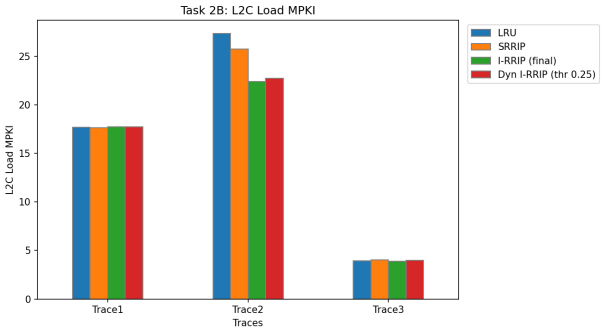
Table 9: Task 2B results. Geometric-mean speedup over LRU: SRRIP 1.0004, I-RRIP 1.0066, Dynamic I-RRIP 1.0060.



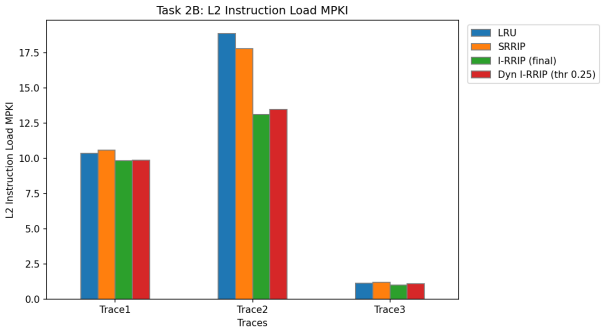
(a) Speedup



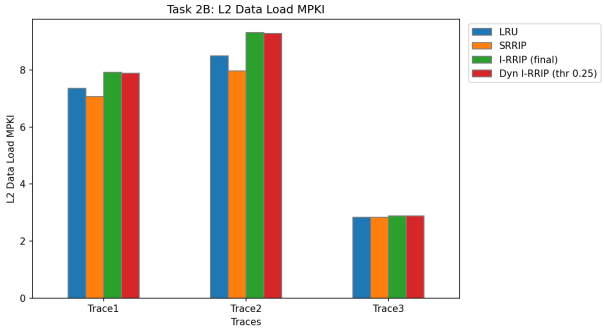
(b) IPC



(c) L2C load MPKI



(d) L2 instruction load MPKI



(e) L2 data load MPKI

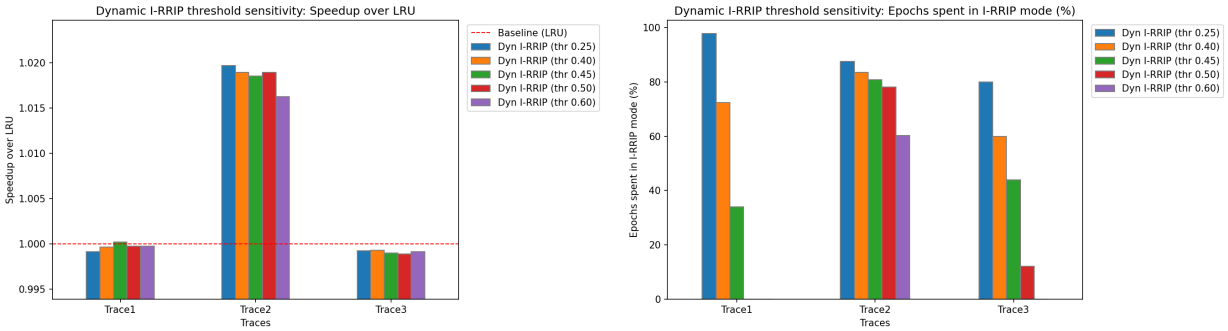
Figure 4: Task 2B plots.

5.3 Sensitivity to the threshold

We use one binary for all five thresholds. The threshold is read from the environment variable `IRRIP_THRESHOLD`. We included the 0.25 from the assignment. We put the other four between 0.40 and 0.60, because that is where the leader ratio of our traces is. At the end of the run it is about 0.46 for Trace1, 0.64 for Trace2 and 0.17 for Trace3, and it changes during the run. A threshold much lower than every ratio would act like 0.25 and tell us nothing new.

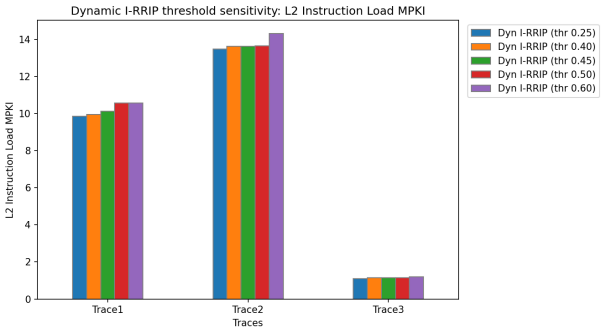
Threshold	T1 speedup	T1 in I-RRIP	T2 speedup	T2 in I-RRIP	T3 speedup	T3 in I-RRIP	Geomean
0.25	0.9991	97.9%	**1.0197**	87.7%	**0.9993**	80.0%	**1.0060**
0.40	0.9996	72.3%	1.0189	83.6%	**0.9993**	60.0%	1.0059
0.45	**1.0002**	34.0%	1.0186	80.8%	0.9990	44.0%	1.0059
0.50	0.9998	0.0%	1.0189	78.1%	0.9989	12.0%	1.0058
0.60	0.9998	0.0%	1.0162	60.3%	0.9991	0.0%	1.0050

Table 10: Threshold sensitivity. “In I-RRIP” is the share of decision epochs in which the followers ran I-RRIP.

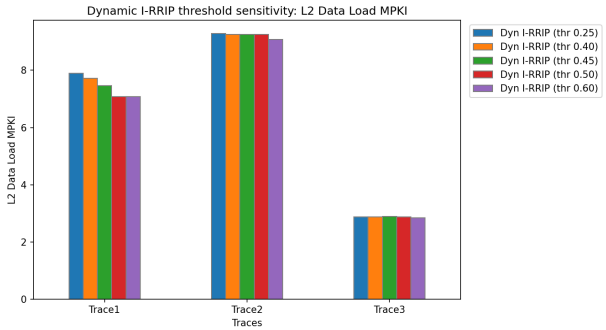


(a) Speedup

(b) Epochs in I-RRIP mode



(c) L2 instruction load MPKI



(d) L2 data load MPKI

Figure 5: Threshold sensitivity plots.

As the threshold goes up, the followers spend fewer epochs in I-RRIP. Instruction MPKI goes back up toward the SRRIP value, and data MPKI goes down toward it. From 0.50 up, Trace1 never uses I-RRIP, and its data MPKI is the same as plain SRRIP (7.076 against 7.077). Trace2 has the highest ratio and stays mostly in I-RRIP even at 0.60. Trace3's ratio changes the most during the run, so its share of I-RRIP epochs drops step by step from 80% to 0%.

We chose 0.25. It has the best geometric mean (1.0060) and the best speedup on Trace2, and it ties with 0.40 for the best on Trace3. On Trace1, 0.45 is better (1.0002) because Trace1 keeps more data lines, but 0.45 loses a little on the other two traces. From 0.25 to 0.50, all differences are about 0.1% of IPC or less. Only 0.60 is clearly worse: it loses 0.35% on Trace2, because it leaves I-RRIP in 40% of that trace's epochs.

5.4 A second design: true set dueling (`dyn_duel.l2c_repl`)

The threshold design uses the traffic ratio to guess whether I-RRIP would help. It never checks whether I-RRIP really misses less. So we also built a policy that measures this directly, using the set-dueling method from class. It still has everything the assignment asks for: leader sets that always run RRIP, and counts of instruction and non-instruction accesses and misses for each leader group. Only the decision is different. `dyn_irrip.l2c_repl` is not changed and gives all the threshold results above.

- Besides the 32 SRRIP leaders (position g in group g), 32 more sets always run I-RRIP. They are at the mirror position, $31 - g$. Since g is never equal to $31 - g$, the two groups never share a set. The other 960 sets are followers.
- PSEL is a 10-bit counter that starts at 512 and stops at 0 and 1023. A miss in an SRRIP leader adds its cost to PSEL. A miss in an I-RRIP leader subtracts it. Followers use I-RRIP while PSEL ≥ 512, which means the SRRIP leaders are missing more. The choice is updated on every leader miss, so we do not need epochs.
- A data miss costs 1 and an instruction miss costs W. This is because an instruction miss stops the front end, while part of a data miss is hidden. W is a power of two, so multiplying by it is

just a shift. We study W instead of a threshold.

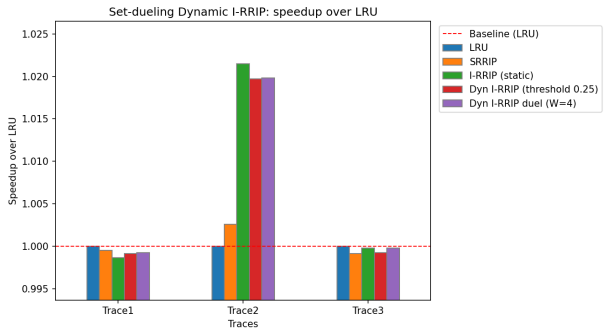
- The decision only needs PSEL and a 5-bit W register: 15 bits, against 67. The cost is 64 leader sets (6.3% of the cache) instead of 32.

Policy	T1 speedup	T2 speedup	T3 speedup	Geomean	Total L2C load MPKI
LRU	1.0000	1.0000	1.0000	1.00000	49.007
SRRIP	0.9995	1.0026	0.9991	1.00041	47.424
I-RRIP (static)	0.9987	**1.0215**	0.9998	**1.00660**	**44.058**
Dynamic I-RRIP, threshold 0.25	0.9991	1.0197	0.9993	1.00598	44.467
Dynamic I-RRIP, set dueling ($W = 4$)	0.9993	1.0198	0.9998	1.00624	44.289

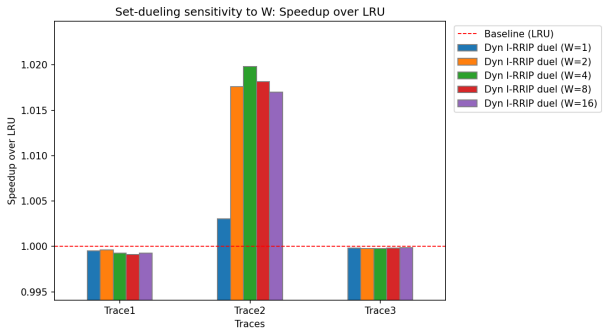
Table 11: Set dueling against the other policies. Total MPKI is summed over the three traces.

W	T1 speedup	T1 I-RRIP fills	T2 speedup	T2 I-RRIP fills	T3 speedup	T3 I-RRIP fills	Geomean	Total MPKI
1	0.9995	2.3%	1.0030	0.0%	0.9998	98.7%	1.00077	47.171
2	0.9996	3.3%	1.0176	38.9%	0.9998	98.9%	1.00564	44.510
4	0.9993	65.0%	**1.0198**	76.9%	0.9998	98.9%	**1.00624**	**44.289**
8	0.9991	83.4%	1.0181	79.8%	0.9998	98.9%	1.00564	44.511
16	0.9992	84.8%	1.0170	76.8%	0.9999	98.8%	1.00534	44.620

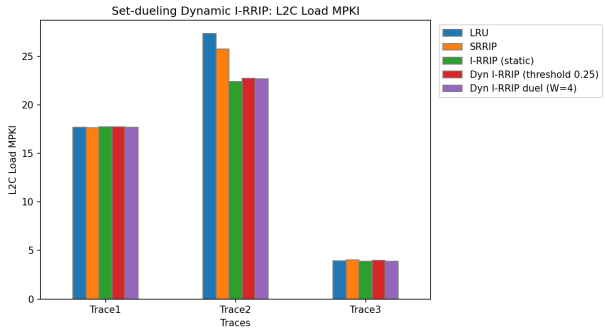
Table 12: Sensitivity to the instruction-miss weight W. "I-RRIP fills" is the share of follower fills made in I-RRIP mode.



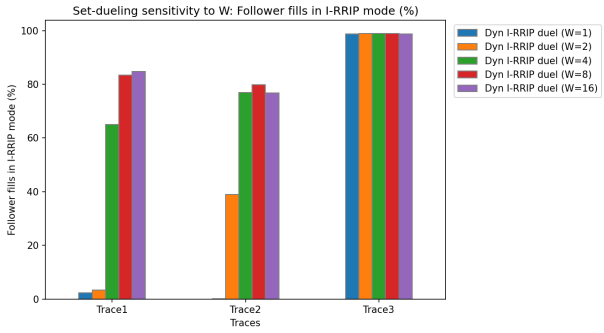
(a) Speedup



(b) Speedup against W



(c) L2C load MPKI



(d) Follower fills in I-RRIP mode

Figure 6: Set dueling plots.

The leader counters show when each trace switches. In the $W = 4$ run, compared with the SRRIP leaders, the I-RRIP leaders:

- on Trace1, save 660 instruction misses and add 2232 data misses;
- on Trace2, save 1823 and add 3868;

- on Trace3, save 1296 and add 60.

The duel picks I-RRIP when W times the saved instruction misses is more than the added data misses. So the break-even weights are 3.4, 2.1 and less than 0.1. The results follow these numbers closely. Trace3 picks I-RRIP for about 99% of fills at every W. Trace2 almost never picks it at $W = 1$, and loses 1.8% to static I-RRIP. It picks I-RRIP for 39% of fills at $W = 2$, and for 77% to 80% from $W = 4$ up. Trace1 is close to its break-even at $W = 4$ and switches mode 252 times. At $W = 16$ each step is so big that it switches 759 times.

We chose $W = 4$. It is the lowest weight above Trace2's break-even, so Trace2 keeps almost all of its gain, while Trace1 stays close to SRRIP. Of the five weights, it has the best geometric mean and the fewest total load misses. The set-dueling version beats the threshold version on every trace (0.9993 against 0.9991, 1.0198 against 1.0197, 0.9998 against 0.9993). It also closes most of the gap to static I-RRIP. On Trace1, the only trace where static I-RRIP is slower than SRRIP, the duel gets back most of SRRIP's performance.

6 Task 3: SPP with prefetch-aware dynamic I-RRIP

6.1 Setup

SPP (Task 1B) runs at the L1D, and dynamic I-RRIP (`dyn_irrip.l2c_repl`) runs at the L2C. The prefetch-aware mode is turned on with the environment variable `IRRIP_PF_AWARE=1`. Without it, the file works exactly as in Task 2B, so the Task 2B results can still be reproduced. The final setup uses threshold 0.55, which is the default in this mode.

Name	L1D prefetcher	L2C replacement
Baseline	none	LRU
SPP	SPP	LRU
Dyn I-RRIP	none	dynamic I-RRIP, Task 2B (threshold 0.25)
SPP + Dyn I-RRIP	SPP	dynamic I-RRIP, Task 2B code, not prefetch-aware (threshold 0.25)
SPP + PF-aware Dyn I-RRIP	SPP	dynamic I-RRIP, prefetch-aware mode (threshold 0.55)

Table 13: Configurations compared in Task 3. The final design must beat SPP and Dyn I-RRIP in IPC.

6.2 How the policy handles prefetch packets

In prefetch-aware mode, four things change.

1. A line brought in by a `PREFETCH` packet gets the highest RRPV. A `PREFETCH` request that hits does not move the line up.
2. A line brought in by a `WRITEBACK` packet is treated the same way. A writeback hit does not move the line up either.
3. The RRPV is 3 bits wide (0 to 7, demand data inserted at 6) instead of 2.
4. The threshold defaults to 0.55 instead of 0.25.

Everything else is the same as Task 2B. Instruction lines still start at 0. Data hits still go to 1 in I-RRIP mode. The victim search still picks data lines at the highest RRPV first. The 32 leader sets and the ratio do not change.

What SPP sends to the L2. SPP sends most prefetches with fill level `FILL_L1`, so the line goes into both the L1D and the L2. Only when the L1D MSHR is more than half full does the line go into the L2 alone. Either way, the L2 sees a packet of type `PREFETCH`. The L2 copy is rarely read, because the demand load finds the line in the L1D. ChampSim's L2 prefetch accuracy shows this: with SPP and LRU it is only 7.7%, 6.9% and 4.7% on the three traces. If a prefetched line is treated like demand data, it takes the place of a line that the core will really ask the L2 for.

Prefetched lines start at the highest RRPV. A prefetched line gets RRPV 7, so it is the first line to be evicted in its set. If a demand load does use it, the normal hit rule moves it up and the line stays. So a prefetch must first prove it is useful in the L2 before it can push out other lines.

Prefetch hits do not move a line up. When a `PREFETCH` request finds its line already in the L2, we do not change the RRPV. A prefetch is only a guess. It is not the core using the line. If we moved the line up, a stream of prefetches could keep a useless line in the cache.

Writebacks are treated the same way. A `WRITEBACK` packet means the L1D is evicting a dirty line. It does not mean the core will read the line again soon. This matters more with SPP. SPP brings extra lines into the L1D, which pushes more dirty lines down to the L2. Without this rule, those writebacks would be inserted and moved up like demand data.

Why 3 RRPV bits. Prefetched and written-back lines sit at RRPV 7. Demand data starts at 6 and has to age more times before it reaches 7. So demand data and page-walk lines stay longer than with 2 bits, while the prefetched and written-back lines still leave first. Page-walk lines gain the most. The policy treats them as data, and each of their misses blocks a load. Compared with SPP + Dyn I-RRIP without prefetch awareness, L2 translation-load MPKI drops from 2.50 to 1.95 on Trace1 and from 0.73 to 0.63 on Trace3. On Trace1 it even ends below the 2.28 of SPP with LRU.

Why threshold 0.55. SPP removes some data misses, so the leaders see a different mix of traffic. At 0.25, Trace1 (leader ratio about 0.45) spends 46 of 48 epochs in I-RRIP, which does not help on that trace. At 0.55, I-RRIP is used where it helps. Trace2 (ratio about 0.63) spends 53 of 75 epochs in I-RRIP. Trace1 and Trace3 (about 0.45 and 0.16) stay in SRRIP mode. The sensitivity study below shows the effect of this choice.

Extra hardware. One more RRPV bit per line: $8192 \times 1 = 8192$ bits (1 KB). Every request already carries its packet type, so we need no new storage to spot `PREFETCH` and `WRITEBACK`.

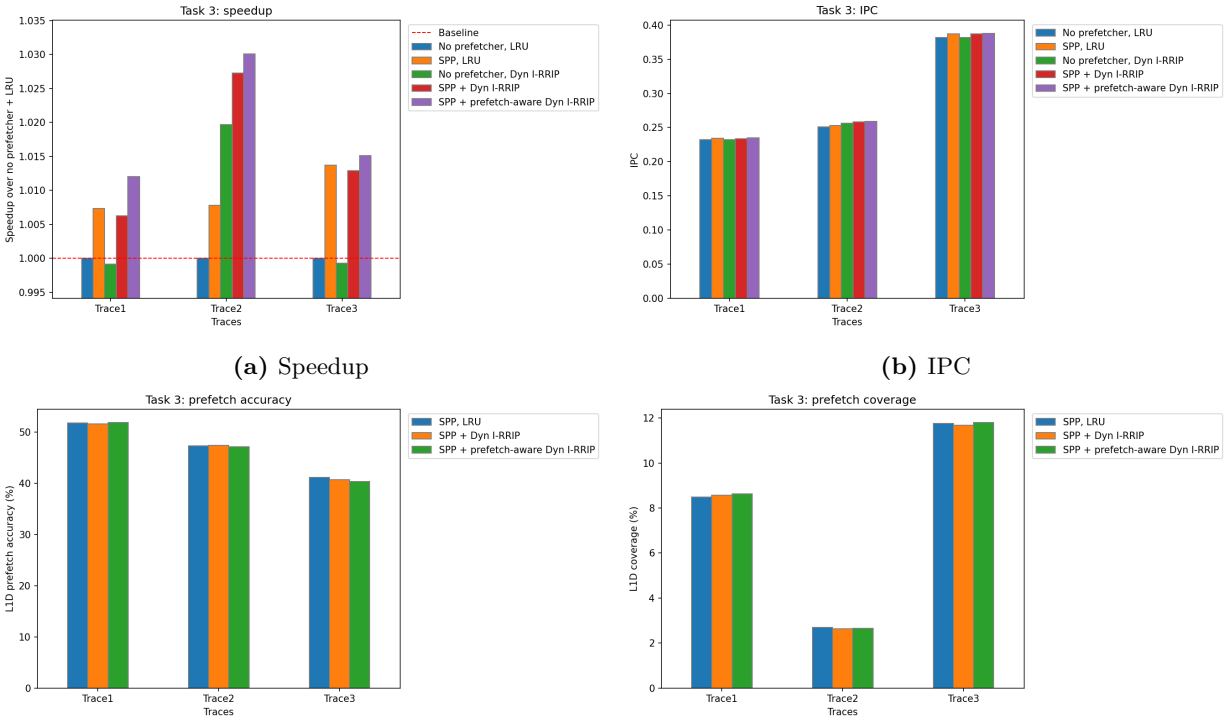
6.3 Results

Trace	Configuration	IPC	Speedup	Acc. (%)	Cov. (%)	L1D MPKI	L2C MPKI	L2 instr.	L2 data
Trace1	Baseline	0.2323	1.0000	n/a	n/a	10.678	17.696	10.338	7.358
	SPP	0.2340	1.0073	51.79	8.49	9.771	16.941	10.379	6.561
	Dyn I-RRIP	0.2321	0.9991	n/a	n/a	10.679	17.740	9.845	7.895
	SPP + Dyn I-RRIP	0.2338	1.0062	51.64	8.57	9.763	16.956	9.907	7.050
	SPP + PF-aware	**0.2351**	**1.0120**	51.89	8.64	9.755	16.940	10.329	6.611
Trace2	Baseline	0.2512	1.0000	n/a	n/a	13.538	27.357	18.853	8.504
	SPP	0.2531	1.0078	47.32	2.69	13.173	27.132	18.950	8.182
	Dyn I-RRIP	0.2561	1.0197	n/a	n/a	13.541	22.752	13.465	9.287
	SPP + Dyn I-RRIP	0.2580	1.0272	47.41	2.63	13.181	22.642	13.707	8.935
	SPP + PF-aware	**0.2587**	**1.0301**	47.18	2.65	13.179	21.590	12.648	8.942
Trace3	Baseline	0.3821	1.0000	n/a	n/a	3.403	3.954	1.124	2.830
	SPP	0.3873	1.0137	41.14	11.76	3.003	3.404	1.137	2.267
	Dyn I-RRIP	0.3818	0.9993	n/a	n/a	3.403	3.976	1.096	2.880
	SPP + Dyn I-RRIP	0.3870	1.0129	40.75	11.68	3.006	3.443	1.123	2.320
	SPP + PF-aware	**0.3878**	**1.0151**	40.35	11.80	3.002	3.330	1.078	2.252

Table 14: Task 3 results. All MPKI columns are load MPKI. Geometric-mean speedup: SPP 1.0096, Dyn I-RRIP 1.0060, SPP + Dyn I-RRIP 1.0154, SPP + PF-aware Dyn I-RRIP 1.0190.

Trace	SPP	Dyn I-RRIP	Final	Final vs SPP	Final vs Dyn I-RRIP
Trace1	0.234046	0.232141	**0.235139**	+0.47%	+1.29%
Trace2	0.253122	0.256103	**0.258708**	+2.21%	+1.02%
Trace3	0.387301	0.381786	**0.387843**	+0.14%	+1.59%

Table 15: IPC of the final configuration against the two designs it has to beat.



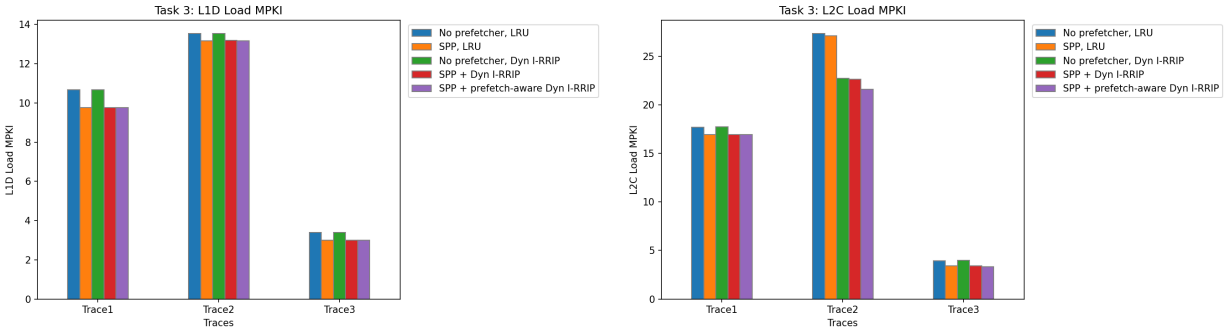
(a) Speedup

(b) IPC

(c) Prefetch accuracy

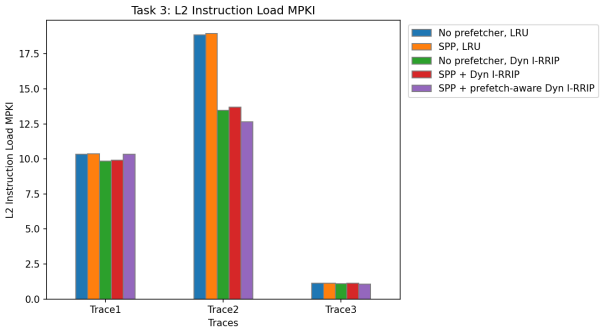
(d) Coverage

Figure 7: Task 3: performance and prefetcher metrics.

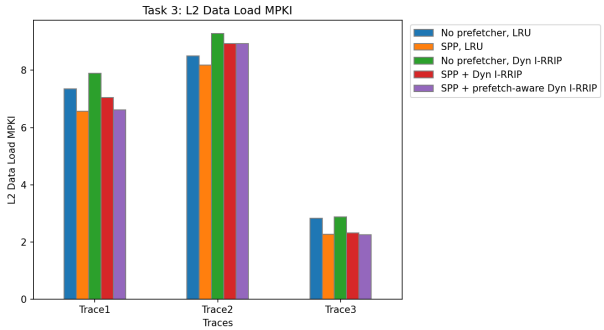


(a) L1D load MPKI

(b) L2C load MPKI



(c) L2 instruction load MPKI



(d) L2 data load MPKI

Figure 8: Task 3: MPKI at each level.

6.4 Sensitivity to the threshold

Threshold	T1 speedup	T1 I-RRIP epochs	T2 speedup	T2 I-RRIP epochs	T3 speedup	T3 I-RRIP epochs	Geomean
0.25	1.0060	46/48	**1.0315**	65/75	**1.0152**	20/26	1.0175
0.45	1.0114	5/48	1.0302	60/75	1.0136	7/26	1.0184
0.50	**1.0120**	0/48	1.0305	58/75	1.0141	2/26	1.0188
0.55	**1.0120**	0/48	1.0301	53/75	1.0151	0/26	**1.0190**
0.60	**1.0120**	0/48	1.0272	38/75	1.0151	0/26	1.0181

Table 16: Threshold sensitivity in prefetch-aware mode (SPP at the L1D in every run).

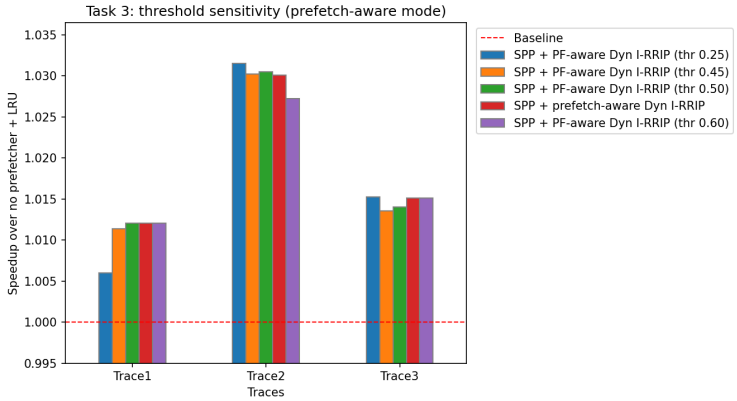


Figure 9: Speedup for each threshold in prefetch-aware mode.

The threshold mainly decides whether Trace1 uses I-RRIP. At 0.25, Trace1 spends almost every epoch in I-RRIP and is slower than SPP alone (IPC 0.233734 against 0.234046). From 0.50 up, it stays in SRRIP mode. Trace2 gains the most from protecting code, so it likes a low threshold. It

starts to lose at 0.60, where it leaves I-RRIP in half of its epochs. At 0.45, Trace3 is just below SPP (0.387256 against 0.387301). Thresholds 0.50, 0.55 and 0.60 beat both SPP and Dyn I-RRIP on every trace. We chose 0.55. It has the highest geometric mean. It also ties with 0.60 for the biggest lead over SPP on Trace3, which is the trace where this lead is smallest.